

HandKey: An Efficient Hand Typing Recognition using CNN for Virtual Keyboard

Avirmed Enkhbat

Department of Computer Science and
Information Engineering
National Central University
Taoyuan, Taiwan
avirmed@g.ncu.edu.tw

Timothy K. Shih

Department of Computer Science and
Information Engineering
National Central University
Taoyuan, Taiwan
tshih@g.ncu.edu.tw

Tipajin Thaisutikul

Department of Computer Science and
Information Engineering
National Central University
Taoyuan, Taiwan
t.greentip@gmail.com

Noorkholis Luthfil Hakim

Department of Computer Science and
Information Engineering
National Central University
Taoyuan, Taiwan
koliskol@gmail.com

Wisnu Aditya

Department of Computer Science and
Information Engineering
National Central University
Taoyuan, Taiwan
wisnuaditya@gmail.com

Abstract— This paper proposes an efficient framework that recognizes hand typing motions and gestures for making a virtual keyboard by using a single RGB camera. There are several works related to virtual keyboard in the Human-computer interaction (HCI) area. Most of them use hand pose estimation, hand shape and external equipment (depth sensor, leap motion, control glove, touch screen etc.). Whereas, our framework does not require additional equipment or prior experience from users, it works like a regular typing action in the air which is similar to typing on a real QWERTY keyboard. It uses convolutional neural networks (CNN) to classify 2 hand typing gestures (touch and non-touch). Also, we train 11 gestures which are non-touch and touching for each 10 fingers of two hands gestures. Proposed CNN model achieves a 99.2% classification accuracy for the 2 gestures case and a 91% classification accuracy for the 11 gestures case.

Keywords— human-computer interaction; hand typing gesture recognition; virtual keyboard; motion history image; convolutional neural network

I. INTRODUCTION

Text entry user interface is the essential part of human interaction access point to trigger the starting, operating, and controlling the processes of the potential system. However, with traditional computing devices, text entry user interfaces still have a bottleneck such as the chance of breakage or infection transfer when it is public usage. Currently, one of the promising approaches to solve this problem is a virtual keyboard. A virtual keyboard is where a full-size image of a QWERTY keyboard is projected onto any surface. Touching the image of any keys will generate the input to the system corresponding to a key's image. With a touchless virtual keyboard, there is no cleaning required, and they have no wires, buttons, or switches to maintain. Those benefit of the virtual keyboard is significant and, therefore, it brought a lot of research attentions into this application area.

In order to provide a robust virtual keyboard, we need an effective hand typing recognition model. Recently, most hand gesture recognition methods focus on tracking hand pose estimation or detecting hand gestures [1]. The hand pose estimation is convenient to track click actions on the clicked region. However, this method is not suitable when the keyboard buttons are too small or there are nearby multiple

buttons. In contrast, our framework can detect typing motions of two-hand fingers in the air which are similar to a regular typing on a keyboard. Therefore, our framework is much easier to use and implement in real-world applications.

In addition to the hand typing recognition model, the data-preprocessing step is also important to enhance the performance. Existing works like a visual keyboard for smart TV in [3] and a virtual keyboard design in [4] proposed hand tracking and fingertip movement recognition methods using geometric features of the hand for data-preprocessing technique. In contrast, we use Motion History Image (MHI) [2] technique for feature extraction. The MHI is widely employed by various researches for action recognition, motion analysis and other related applications. The advantage of MHI is showing an object movement, clear shapes and removing the background, as shown in Figure 1 [2].

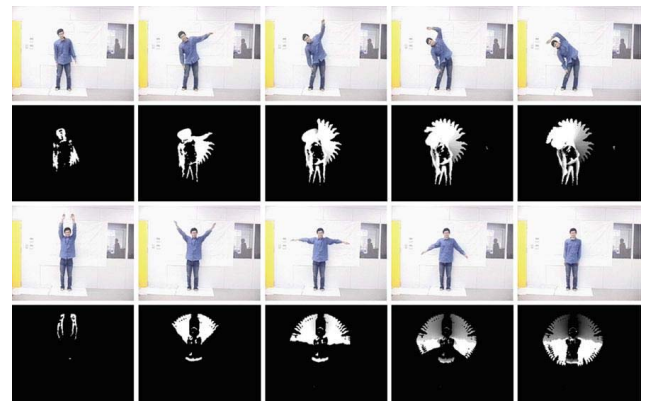


Figure 1. Motion History Image

In Figure 1, the first and third rows show two different actions sequentially. The bottom row presents MHIs (1st and 3rd) for sideways body-bending and two-hand waving actions. A benefit of using the gray-scale MHI is that it is sensitive to the direction of motion and better suited for discriminating between actions of opposite directions. A hand typing gesture recognition technology was developed based on Convolutional Neural Network (CNN) [6]. CNN is heavily used for image classification, gesture recognition and object detection applications of machine learning. Motivated by the achievement of MHI and CNN techniques, we propose a

framework that integrates both techniques named “HandKey”. With Handkey, we not only solve several issues related to hand positions, camera distances, and light effects, but we also provide the feasible and easy to use virtual keyboard application as a public trail to the research community.

First, we fix an area for the input which enables us to detect finger touching in the center of the area. Then, we determine other fixed input areas for the remaining buttons on the input image. Figure 2 shows the fixed input area for the touched button in the white rectangle and other determined input areas for buttons on a keyboard. Note that green rounded rectangles resemble keyboard buttons. In this paper, we consider the standard QWERTY keyboard.

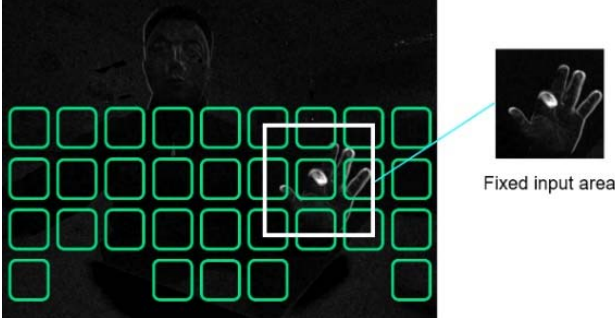


Figure 2. Multiple inputs on input data

II. RELATED WORK

Most hand gesture recognition systems detect hand pose estimation to track hand landmark. When a landmark of fingertip and hand position move in the button area, it triggers a click action. Also, some special gestures are used to detect a click action. For example, fingertip holding a few seconds on the button, performing index finger, clenched fist and fingertip touching on hand palm are used [3-6]. These approaches can detect exact clicking, reduce errors and do not require additional equipment except a camera. However, some of them require continuous clicking time or hand gestures to perform. [4, 7] propose new keyboard layout designs, which substantially improve the typing speed by cutting down unnecessary hand movements during typing.

T. Lee et al. [4] produced ambidextrous keyboard design which is proposed new keyboard layout and identified the type-in action. The new keyboard layout is optimized frequency of key tapping is as close as characters. The type-in gesture is contact between the thumb and the index finger. The index finger points to the input character and remains still the same position while the contact of the thumb to the index finger makes the type-in action.

III. PROPOSED METHOD

Detecting the gesture to click on a button by a single RGB camera is a complicated task. We tested several different methods for detecting to click action based on finger positions, angles and movement speed. But these methods had many issues that cannot be solved smoothly. Finally, we have chosen hand typing motion for the detection.

In this work, we accomplished an efficient detecting method for typing action using basic CNN. The output classifications of CNN are touch and non-touch. The overall architecture of our proposed method is shown in Figure 3.

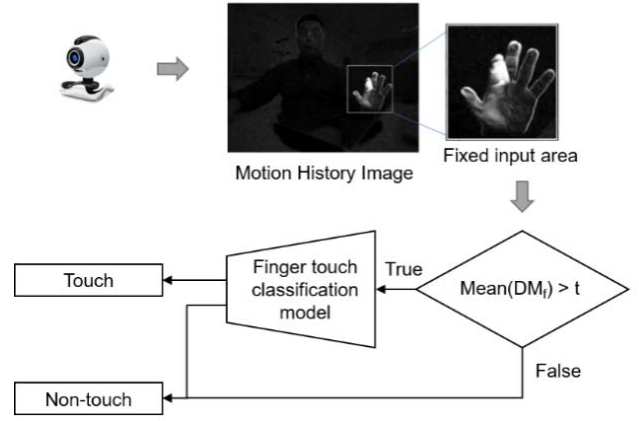


Figure 3. Proposed architecture

A. Data preprocessing

To recognize the typing action, we need hand motion shapes on image sequences from a single RGB camera first. For this, we use the Background Subtraction Technique (BST), a widely used approach for detecting moving objects in video frames from static cameras. The main idea of the technique is detecting the moving objects from the difference between the current frame and a reference frame. We have tested several BST such as Gaussian Mixture-based Background, Foreground Segmentation Algorithm, and Bayesian segmentation. However, they could not show enough results. Then we have derived a BST for MHI by modifying existing BST. Our MHI has three following steps.

First step. To convert the RGB image to greyscale image.



Figure 4. A greyscale image sequence

The image sequence is obtained from the last frames of the camera as shown in Figure 4.

Second step. To get the difference map from consecutive two images as shown in Figure 5.



Figure 5. Difference maps

Third step. To generate a final difference map by overlapping all the difference maps. The final difference map is shown in Figure 6.

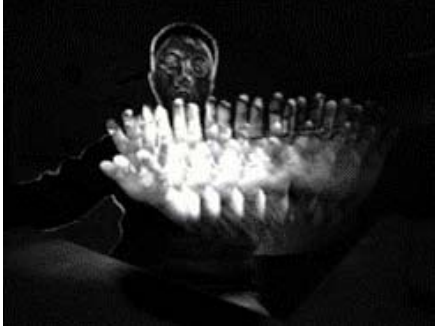


Figure 6. Final difference map (MHI)

$$M_f = I_f - I_{f-1}, f = \{1 \dots n\} \quad (1)$$

Here, n is a number of the images in the sequence. In the experiment, we defined n is equal to 15.

$$DM = \sum_{i=0}^n \frac{M_i}{(n-i)} \quad (2)$$

Here, DM is the overlapped difference map.

Finally, dilation and erosion techniques are applied to the final image for making clearer, as shown in Figure 7 and Figure 8.



Figure 7. Touch dataset

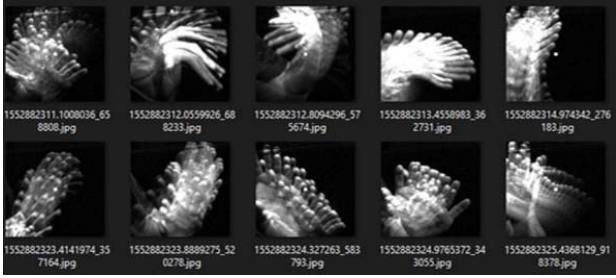


Figure 8. Non-touch dataset

B. Data collection

Data collection is one of the important parts of building a deep learning model. Since our method is different from the existing methods, there is no public dataset suitable for our model. Therefore, we develop a simple data collection tool to speed up our data collecting process. The tool collects image sequences for each class. Then image sequences were generated to get a single image of MHI. A screenshot of the data collection tool is shown in Figure 9.

Our dataset contains 5,180 data collected by 12 people. The data in the dataset is classified into 11 classes in terms of finger motions. There is one class for the negative case and other ten classes are for 10 fingers of the two hands. The negative case consists of random hand motions, expect touch actions, as shown in Figure 8.



Figure 9. Data collection tool

We train the model with two different labels. In the first model, the classifications are touch or non-touch. In the second model, the classifications are all 11 classes as shown in Figure 10. It enables us to determine which finger touches.



Figure 10. Finger classes.

Accuracies and losses of the trained two models are shown in Section 4.

C. CNN models

Our proposed model is inspired by VGG16 architecture, as shown in Figure 11. The CNN model should not have many hidden layers, otherwise it affects the speed of our framework. For example, if a virtual keyboard has 40 buttons and hand moves on all buttons in one frame, it must predict all 40 buttons same time, then the frame rate slows down lower than 15fps. For that reason, we chose a shallow deep learning model.

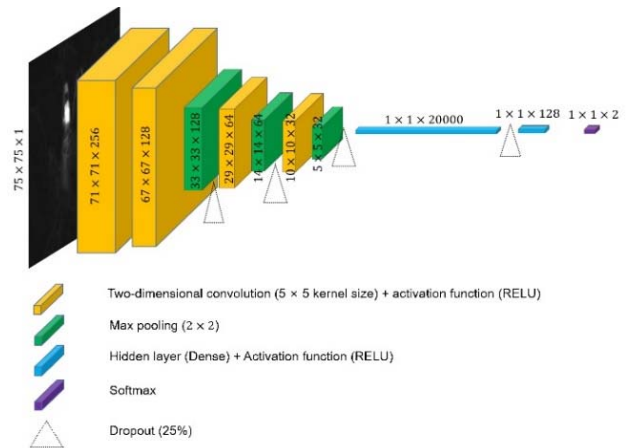


Figure 11. Proposed CNN model

In Figure 11, the input of CNN is a grayscale image and the output label is 0 or 1 which represents a touch or non-touch action, respectively.

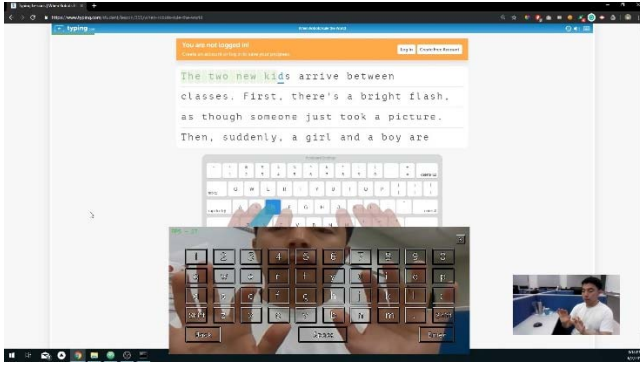


Figure 12. A performer is typing on Virtual Keyboard

IV. EXPERIMENTAL RESULT

We trained the model by our dataset which contains 5,180 data collected by 12 people. Since we have two models, there are two training results. The first result is labelled finger touch and not touch on the button, as shown in Figure 13, Table I. The second result is finger not touch and each ten fingers touch on the button as shown in Figure 14, Table II.

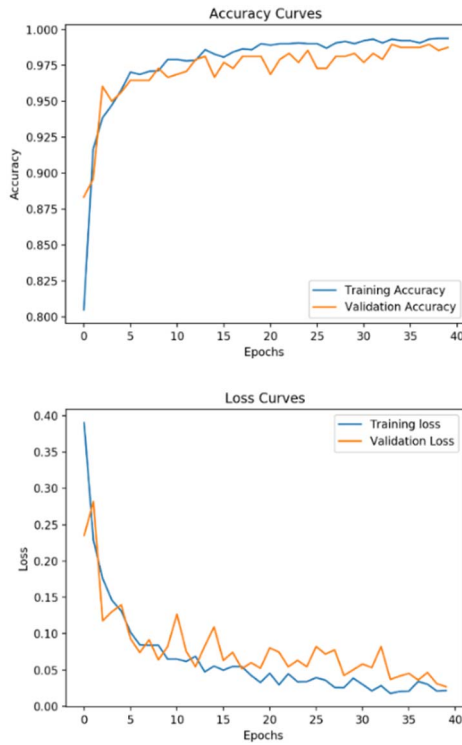


Figure 13. Training result of the proposed model with 2 classes

TABLE I. RESULT OF 2 CLASSES

Training accuracy	Training loss	Validation accuracy	Validation loss
0.992%	0.033%	0.991%	0.064%

We have tested our virtual keyboard in real-time with three groups of people as shown in Figure 12. The people are grouped into not experienced, less experienced, and experienced groups. In this case, experience produces how experienced these groups of people have previously worked on a similar system. The experienced group includes our team members, they know how to use our framework and much times typing on the virtual keyboard. The people of not

experienced were first time tried to type on the virtual keyboard.

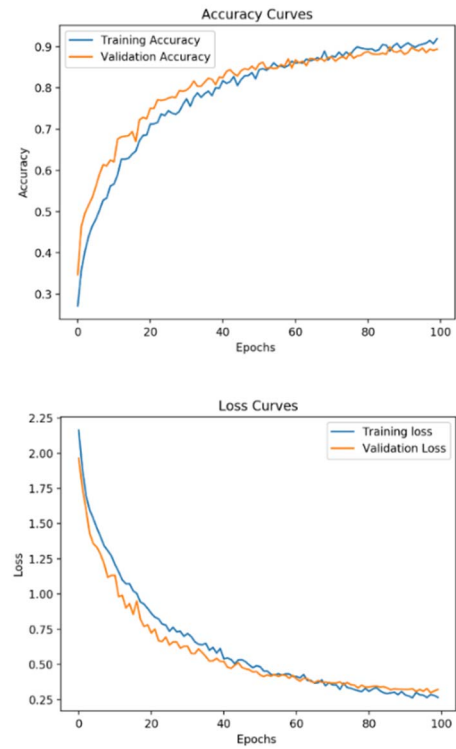


Figure 14. Training result of the proposed model with 11 classes

TABLE II. RESULT OF 11 CLASSES

Training accuracy	Training loss	Validation accuracy	Validation loss
0.91%	0.284%	0.889%	0.365%

We evaluated the testing results by giving scores according to the below equation.

$$Score_{precision} = \frac{Total_{click}}{Total_{click} + Miss_{click} + Error_{click}}$$

Here, Miss click is the number of the wrong predictions on the buttons, Error click is the number of the undetected clicks, Total click is the number of all clicks made by a person in the group. The experiment result is shown in Table III. During the experiment, all three groups type the following paragraph.

Paragraph: *The Moon is a barren, rocky world without air and water. It has dark lava plain on its surface. The Moon is filled with craters. It has no light of its own.*

Paragraph length: 32 words, 155 characters (including space).

TABLE III. TESTING VIRTUAL KEYBOARD IN REAL TIME

	Miss	Error	Time (min.sec)	Score
Not experienced	64	13	12.16	0.6681
Less experienced	35	26	10.50	0.7175
Experienced	12	2	3.32	0.9171

CONCLUSION

In this paper, we present an efficient hand typing motion recognition system for Virtual Keyboard using a single RGB camera. The recognition part uses MHI for feature extraction and classify data using CNN. Our virtual keyboard runs smoothly when FPS is 25-30.

In real-time testing, the best score is 92% in the experienced group. In the not experienced group, the score is 67%. The two scores were quite a difference because not experienced group first time type in the air and it needs more practice. Also, they understand to perform air typing quickly. It shows our framework is ready to be used for real world applications. As future work, we plan to add push clicking gesture in order to improve real-time performance and apply for mobile devices.

Demo video: https://youtu.be/_7Cl-yaPhRs

REFERENCES

- [1] Y. Zhou, G. Jiang, Y. Lin, A novel finger and hand pose estimation technique for real-time hand gesture recognition, *Pattern Recognition*, January 2016
- [2] Md. Atiqur Rahman Ahad, T. Jie, H. Kim, S. Ishikawa, Motion history image: Its variants and applications, *Machine Vision and Applications*, 2012, pp. 255–281
- [3] E. Togootogtokh, A. Amartuvshin, R. Huang, Visual Keyboard for Smart TV using RGB camera, *Ubi-Media Computing*, 2014
- [4] T. Lee, H. Lee, Ambidextrous Virtual Keyboard Design with Finger Gesture Recognition, *ISCAS*, 2018
- [5] Y. Zhang, W. Yan, A. Narayanan, A virtual keyboard implementation based on finger recognition, *IVCNZ*, 2017
- [6] R. Patel, J. Dhakad, K. Desai, T. Gupta, S. Correia, Hand Gesture Recognition System using Convolutional Neural Networks, *ICCCA*, 2018
- [7] M. Chang, T. Chiueh, C. Chang, Virtual Keyboard for Head Mounted Display-based Wearable Devices, *ICPADS*, 2014
- [8] Y. Zhang, W. Yan and A. Narayanan, A Virtual Keyboard Implementation Based on Finger Recognition, *IVCNZ*, 2017
- [9] A. Henzen, P. Nohama, Adaptable virtual keyboard and mouse for people with special needs, *FTC*, 2016
- [10] J. Shin, C. Kim, Non-Touch Character Input System Based on Hand Tapping Gestures Using Kinect Sensor, *IEEE*, 2017